Multi-Agent Database Analyzer

Architecture, System Design and Agentic Chat Support

Sujeet Banerjee

This handout presents the architecture and system design of a multi-agent Database Analyzer that allows users to ask natural-language questions about large databases and receive evidence-based summaries, analysis, and follow-up insights. The system combines agentic orchestration, schema and metadata retrieval, Text-to-SQL, query validation, database-side computation, statistical analysis, and an LLM-based conversational interface. A central design principle is that the LLM should not attempt to ingest an entire database. Instead, the database performs the heavy computation and the agents coordinate what should be queried, analyzed, and investigated. This architecture makes the approach more scalable, safer, and more auditable.

Architecture at a Glance

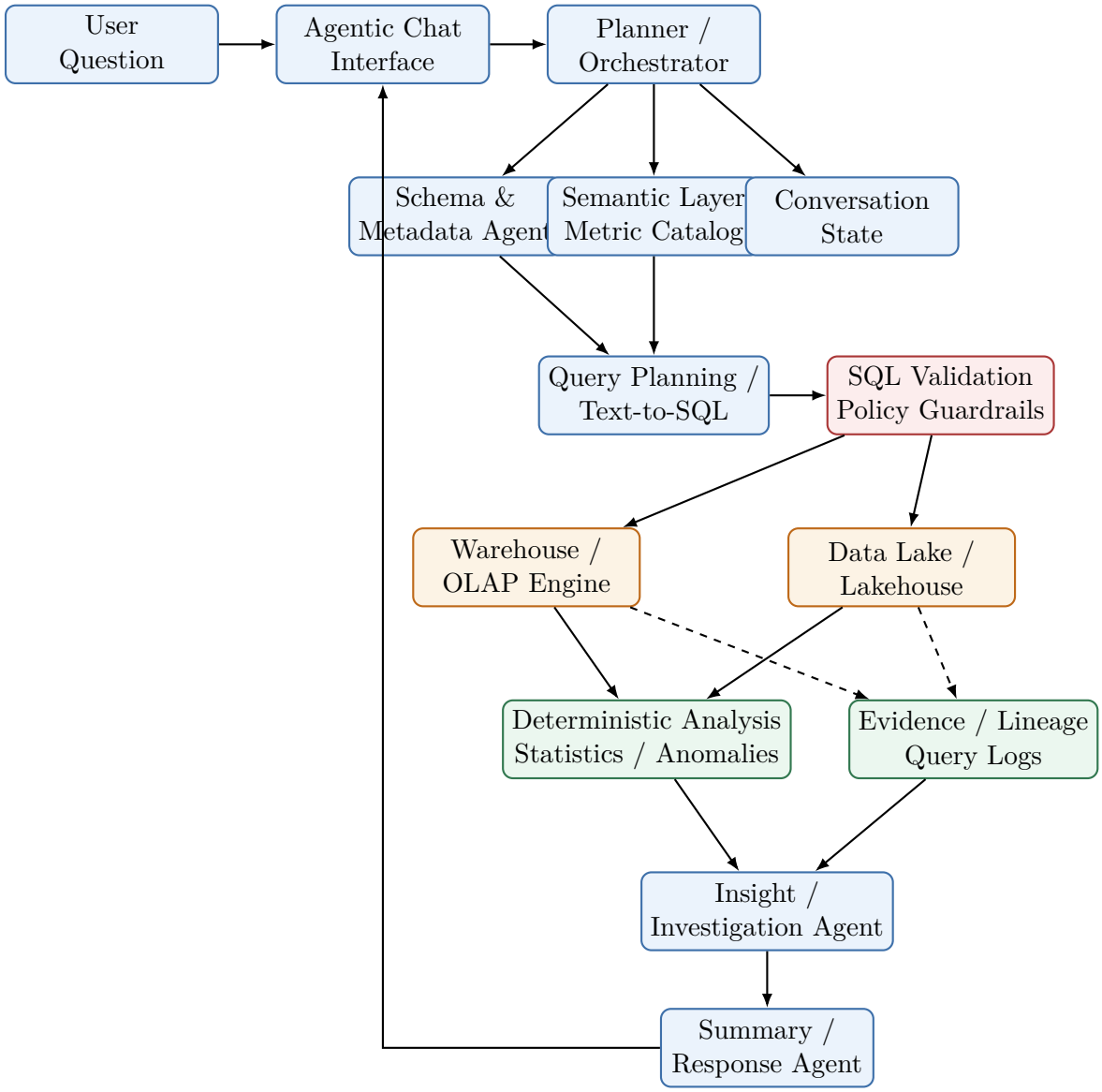

Figure 1: Layered reference architecture.

Core principle: the LLM plans and interprets; the data platform performs large-scale computation and provides evidence.

1 Architecture Components

The architecture separates the system into logical responsibilities. These responsibilities may initially run in one application process and later be split into independent agents or services.

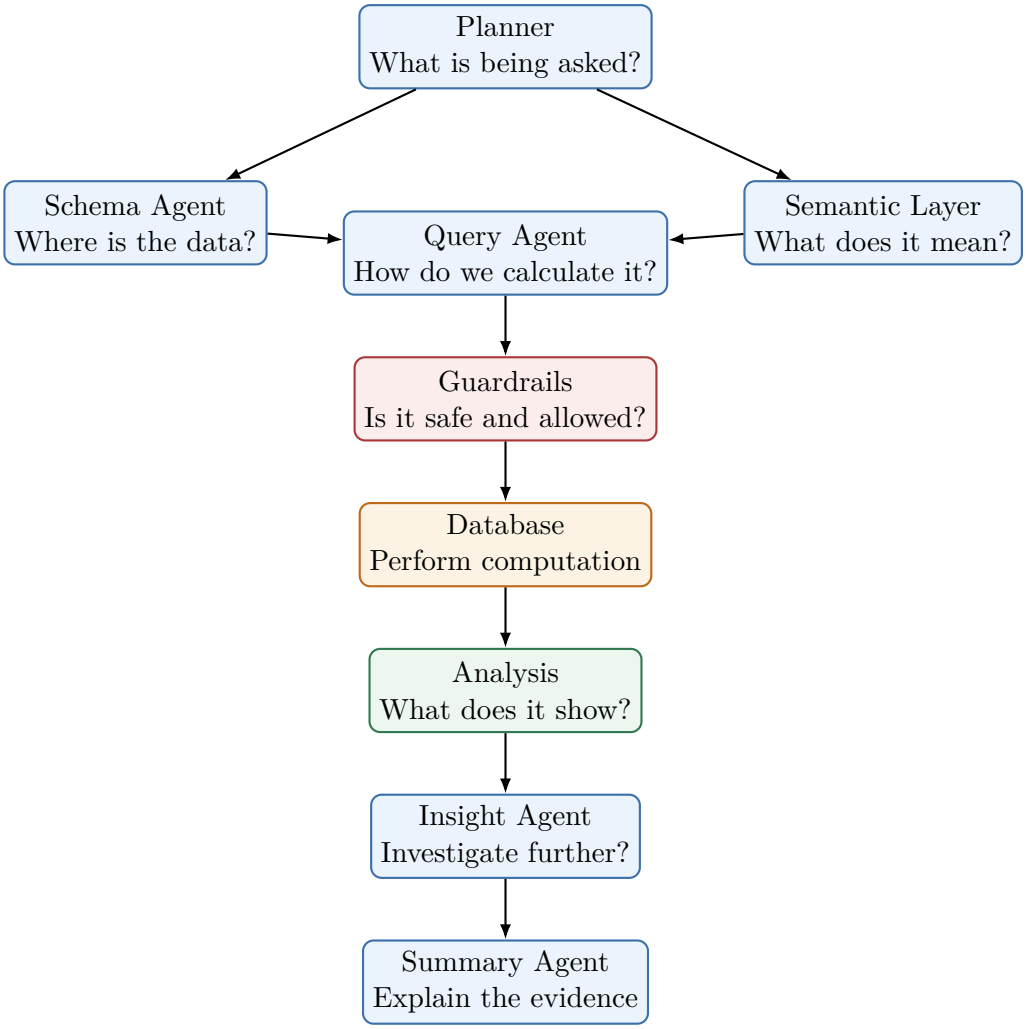

Figure 2: Separation of agent responsibilities.

2 Agentic Investigation Loop

A fixed Text-to-SQL pipeline answers one question. An agentic analyzer can continue investigating when an intermediate result suggests a useful drill-down.

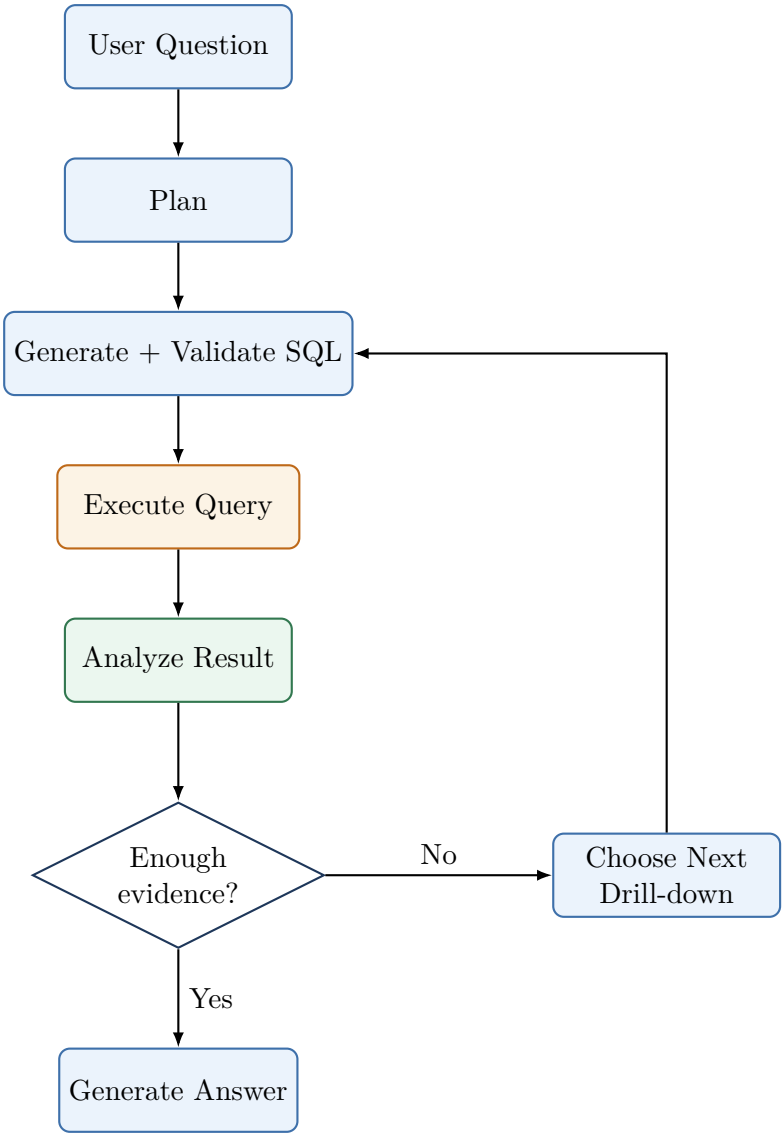

Figure 3: Bounded agentic investigation loop.

The loop should be bounded by maximum steps, query count, execution time, estimated cost, and result size.

3 Schema RAG and Semantic Layer

For a database containing thousands of tables, the LLM should retrieve only the relevant metadata.

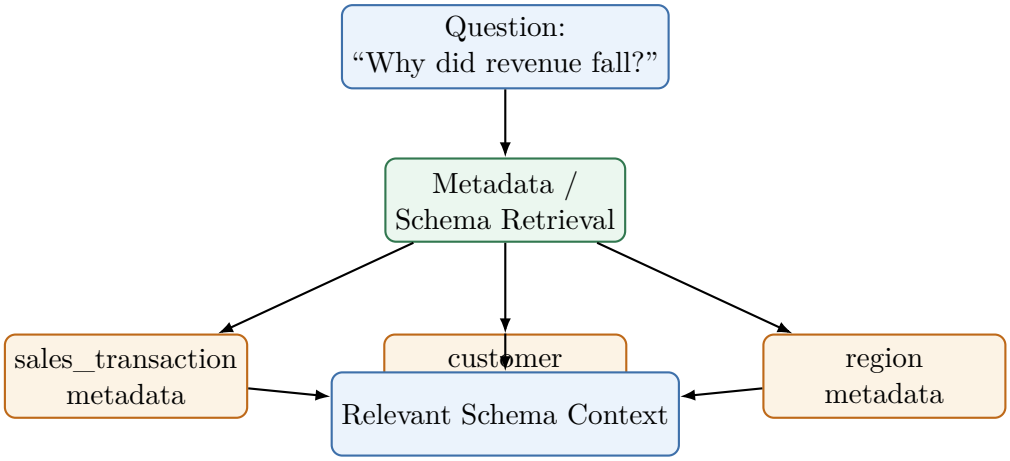

Figure 4: RAG-style retrieval over database metadata.

The semantic layer then resolves business concepts such as “revenue” into certified definitions before SQL generation.

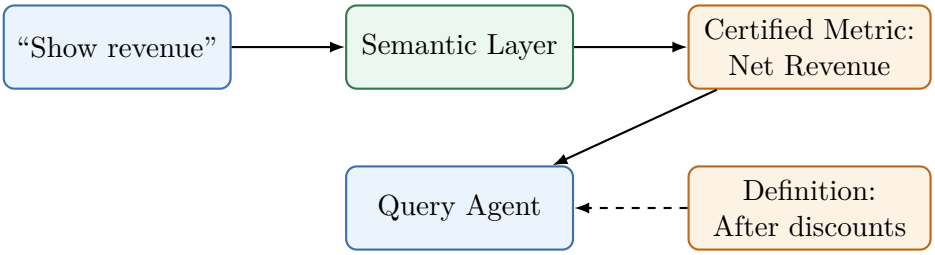

Figure 5: Business semantics before SQL generation.

4 Safe Text-to-SQL Pipeline

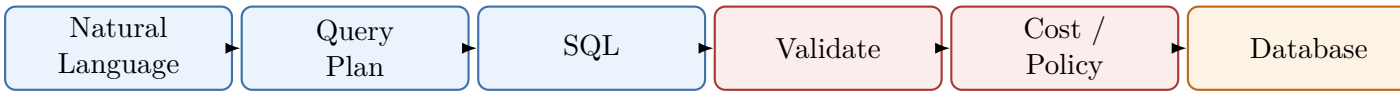

Figure 6: Controlled Text-to-SQL execution path.

The validation layer should enforce read-only access, permitted tables and columns, required filters, join safety, estimated query cost, and timeouts.

5 Large-Scale Data Flow

The LLM should not receive billions of raw records.

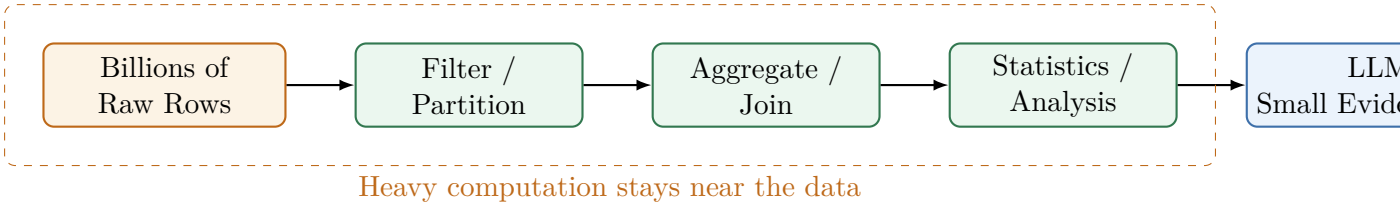

Figure 7: Database-side computation before LLM interpretation.

Useful techniques include partitioning, columnar storage, materialized views, pre-aggregation, OLAP engines, lakehouses, caching, parallel queries, and resource quotas.

6 Hierarchical Summarization

For extremely large datasets, summaries can be produced at increasing levels of aggregation.

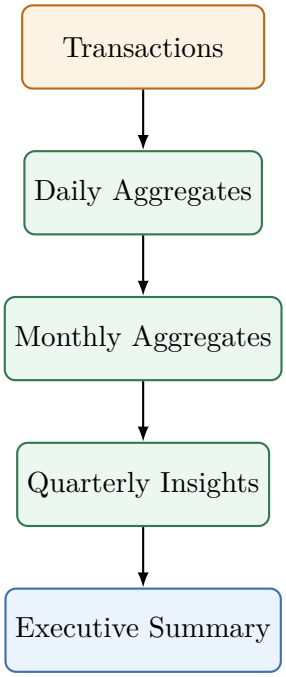

Figure 8: Hierarchical summarization.

This resembles a MapReduce pattern: analyze smaller units first, then combine the resulting summaries.

7 Evidence and Explainability

A generated insight should be connected to the evidence that supports it.

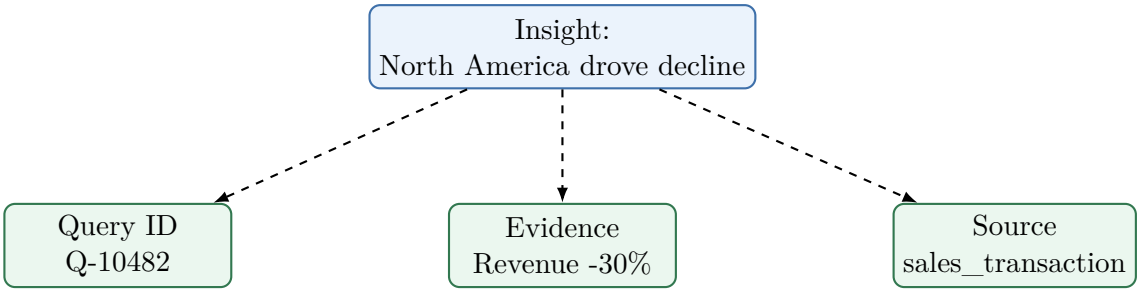

Figure 9: Evidence binding for explainability and auditability.

This enables query lineage, reproducibility, debugging, audit trails, and greater user trust.

8 Security and Governance

Authorization should not be delegated to the LLM.

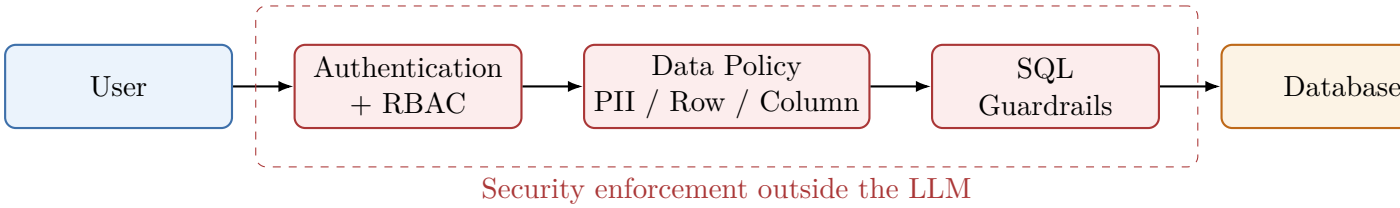

Figure 10: Defense-in-depth security controls.

9 End-to-End Example

For the question "Why did revenue decline in Q2?", the system may perform:

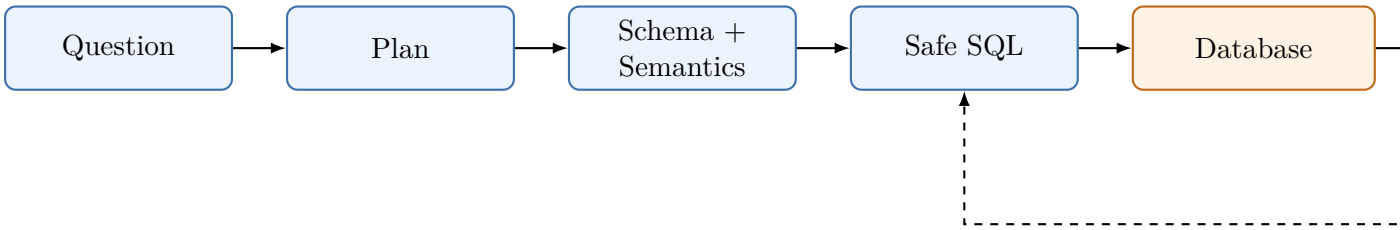

Figure 11: Simplified end-to-end investigation.

A possible evidence chain is:

1. Revenue declined by 12%.
2. Region analysis identifies North America as the major contributor.

3. North America declined by 30%.
4. Product analysis identifies Product X as the major contributor.
5. Product X explains approximately 65% of the regional decline.
6. Customer-segment analysis identifies enterprise customers as the largest affected segment.

10 Recommended Student POC Roadmap

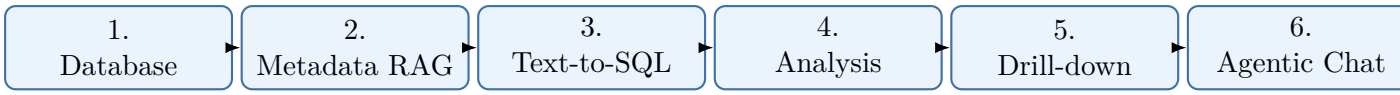

Figure 12: Incremental POC roadmap.

11 Design Principles

1. LLM as planner and interpreter, not the database engine.
2. Push computation to the data platform.
3. Retrieve relevant schema instead of exposing all tables.
4. Use a semantic layer for business meaning.
5. Validate SQL before execution.
6. Use deterministic tools for numerical calculations.
7. Bound agentic loops with explicit budgets.
8. Keep authorization outside the LLM.
9. Record evidence and lineage.
10. Return conclusions with supporting evidence.
11. Start with a controlled POC before introducing full autonomy.

12 Discussion Questions

1. What changes if the database contains 10 billion rows?
2. How can the agent find the right table among 10,000 tables?
3. How should “revenue” be resolved when multiple definitions exist?
4. How can expensive full-table scans be prevented?
5. Should an agent ever execute arbitrary SQL?
6. How should PII and row-level access be protected?
7. What happens when two agents produce contradictory findings?

8. How should SQL accuracy and answer faithfulness be evaluated?
9. When should an autonomous investigation stop?
10. Which operations should be deterministic rather than LLM-generated?

13 Key Takeaway

Database → LLM → Summary

is a poor architecture for large-scale data analysis.
A more robust architecture is:

User → Planner → Metadata / Semantics → Safe Query → Database → Analysis → Evidence → LLM Summ

Think of the LLM as the analyst's brain,
but the database as the analyst's calculator and evidence base.

Contents

14 Problem Statement

Consider an organization with a large analytical database containing customers, orders, products, transactions, regions, support interactions, and other operational data.

A user may ask:

> "Summarize our business performance last quarter. Why did revenue decline, and which products and customer segments caused the decline?"

A conventional chatbot cannot simply retrieve every row and send it to an LLM. The database may contain millions or billions of rows.

The desired system therefore needs to behave more like an intelligent analyst:

1. Understand the user's question.
2. Identify the relevant business concepts and metrics.
3. Discover the relevant database schema.
4. Formulate an analytical plan.
5. Generate safe and efficient database queries.
6. Execute computation close to the data.
7. Inspect the results.
8. Decide whether additional investigation is required.
9. Produce a concise, evidence-based explanation.
10. Support follow-up questions while preserving conversation context.

The key architectural idea is:

> **The agent should not read the database; the agent should decide what the database needs to calculate.**

15 System Goals

The proposed system should provide:

- Natural-language access to structured data.
- Safe Text-to-SQL generation.
- Metadata-aware schema discovery.
- Business-semantic understanding of metrics.
- Scalable analysis of very large datasets.
- Multi-step investigative reasoning.
- Conversational follow-up.
- Query and reasoning traceability.
- Access control and data protection.
- Clear separation between computation and language generation.

16 High-Level Architecture

Figure 13 shows the conceptual architecture.

17 Why a Multi-Agent Architecture?

A single LLM prompt can perform simple Text-to-SQL tasks, but a production system needs several distinct responsibilities.

For example:

Component	Primary responsibility
Planner	Converts the user request into an analytical plan.
Schema Agent	Finds relevant tables, columns, relationships, and metadata.
Semantic Layer	Resolves business meanings such as "revenue" or "active customer".
Query Agent	Generates analytical SQL or query plans.
Guardrail Agent	Validates SQL, permissions, cost, and safety constraints.
Analysis Agent	Computes trends, comparisons, anomalies, and statistical findings.
Insight Agent	Decides whether additional drill-down is necessary.
Summary Agent	Converts evidence into a readable explanation.
Memory	Maintains conversational context and prior analytical state.

These components do not necessarily need to be separate LLM instances. They are logical responsibilities that can initially be implemented using a single orchestrator and progressively separated as the system grows.

18 Agentic Chat Support

The chat interface is more than a Text-to-SQL interface.

Suppose the user asks:

> "What happened to revenue last quarter?"

The system might first discover:

1. Revenue decreased by 12%.
2. North America contributed most of the decline.
3. Product X explains most of the North American decline.
4. Enterprise customers experienced the largest reduction.

The user can then ask:

> "Why did Product X decline?"

The system should not start from scratch. It should retain analytical state and understand that "Product X" refers to the previously identified product.

A useful conversation state can include:

```
{
  "time_period": "previous quarter",
  "metric": "net_revenue",
  "analysis_context": "revenue_decline",
  "identified_region": "North America",
  "identified_product": "Product X",
  "previous_queries": [...],
  "evidence": [...]
}
```

This enables conversational investigation.

19 Agentic Investigation Loop

The key difference between a fixed pipeline and an agentic system is the ability to decide whether more analysis is needed.

The loop must be bounded. Otherwise, an autonomous agent can generate unnecessary queries and consume significant database resources.

Useful controls include:

- Maximum number of reasoning steps.
- Maximum number of database queries.
- Query timeout.
- Maximum estimated query cost.
- Maximum rows returned to the LLM.
- Allowed tables and columns.
- User-specific access policies.

20 Schema and Metadata Design

Schema discovery is one of the most important components of the system.

A large enterprise database may contain thousands of tables. Supplying the entire schema to an LLM is inefficient.

Instead, maintain a metadata catalog.

A useful metadata record might contain:

```
{
  "table": "sales_transaction",
  "description": "Completed customer sales transactions",
  "business_domain": "Sales",
  "owner": "Finance",
  "certified": true,
  "columns": [
    {
      "name": "transaction_date",
      "description": "Date of completed transaction",
      "semantic_type": "date"
    },
    {
      "name": "customer_id",
```

```
      "description": "Unique customer identifier",
      "semantic_type": "identifier"
    },
    {
      "name": "net_revenue",
      "description": "Revenue after discounts",
      "semantic_type": "currency"
    }
  ]
}
```

The schema metadata can be indexed for semantic retrieval.
For example:

$$\text{User Question} \rightarrow \text{Metadata Retrieval} \rightarrow \text{Relevant Tables} \rightarrow \text{Relevant Columns}$$

This is effectively **RAG over database metadata**.

21 Semantic Layer and Business Metrics

Technical schema alone is not enough.
Consider the word "revenue". It may refer to:

- Gross revenue.
- Net revenue.
- Recognized revenue.
- Booked revenue.
- Recurring revenue.
- Cash collected.

A semantic layer can define the organization's approved business metrics.
For example:

```
Metric: Net Revenue

Definition:
Revenue recognized after discounts and adjustments.

Formula:
SUM(net_revenue)

Source:
finance_revenue

Owner:
Finance

Certified:
Yes
```

This significantly reduces the risk that the LLM generates technically valid but business-incorrect SQL.

22 Query Generation and Validation

The Query Agent should not be allowed to execute arbitrary SQL.

A recommended pipeline is:

Natural Language → Query Plan → SQL → Validation → Cost Check → Execution

Validation should check:

- SQL syntax.
- Allowed tables.
- Allowed columns.
- User permissions.
- Prohibited operations.
- Missing filters.
- Cartesian joins.
- Excessive scan risk.
- Query timeout.
- Estimated cost.

For analytical workloads, read-only database credentials should normally be used by the agent.

23 Scaling to Very Large Databases

The most important scaling principle is:

Move computation to the data rather than moving the data to the LLM.

The system should avoid:

```
SELECT * FROM billion_row_table;
```

Instead, it should push computation into the database:

```
SELECT
    region,
    SUM(net_revenue) AS revenue,
    COUNT(DISTINCT customer_id) AS customers
FROM sales_transaction
WHERE transaction_date >= DATE '2026-04-01'
  AND transaction_date < DATE '2026-07-01'
GROUP BY region;
```

The LLM receives the small aggregated result rather than the raw records.

23.1 Scaling Techniques

Important techniques include:

- Partitioning by time or business dimensions.
- Indexing where appropriate.
- Columnar storage.
- Materialized views.
- Pre-aggregated fact tables.
- OLAP warehouses.
- Data lakehouse architectures.
- Query result caching.
- Approximate algorithms for expensive distinct counts.
- Parallel analytical queries.
- Workload management and resource quotas.

24 Hierarchical Summarization

For very large datasets, analysis can be hierarchical.
For example:

Transaction Data → Daily Aggregates → Monthly Aggregates → Quarterly Insights → Annual Summary

This resembles a MapReduce pattern:

Map: Analyze smaller units

Reduce: Combine the resulting summaries

The LLM should generally operate on the smaller, higher-level analytical representation.

25 Data Analysis and Insight Detection

The Data Analysis Agent can perform deterministic calculations before the LLM sees the data.
Examples include:

- Percentage change.
- Moving averages.
- Trend detection.
- Variance analysis.
- Outlier detection.
- Correlation analysis.

- Distribution analysis.
- Segment comparisons.

For example, if quarterly revenue changes from $120M$ to $138M$:

$$Growth = \frac{138 - 120}{120} \times 100 = 15\%$$

The LLM should not be responsible for doing every numerical calculation itself. Deterministic tools should calculate the numbers; the LLM should interpret them.

26 Evidence and Explainability

Every generated insight should ideally have supporting evidence.

A response could maintain:

```
Insight:
North America caused most of the revenue decline.

Evidence:
Query ID: Q-10482
Revenue change: -30%
Contribution to total decline: 72%

Source tables:
sales_transaction
customer
region
```

This provides a foundation for:

- Auditability.
- Debugging.
- User trust.
- Reproducibility.
- Governance.

The final answer should distinguish between:

1. Observed facts.
2. Statistical findings.
3. Agent-generated interpretations.
4. Hypotheses requiring further investigation.

27 Security and Governance

A database agent is potentially a high-risk system because it can provide access to sensitive enterprise data.

Security must therefore be enforced outside the LLM.

Important controls include:

- Authentication.
- Role-based access control (RBAC).
- Row-level security.
- Column-level security.
- PII masking.
- Data classification.
- Read-only credentials.
- Query allowlists and denylists.
- Query logging.
- Audit trails.
- Rate limits.
- Cost limits.

A critical principle is:

Never rely on the LLM to enforce authorization.

The database and policy enforcement layers must independently enforce access restrictions.

28 Major System Bottlenecks

Bottleneck	Why it matters	Possible solution
Context size	Entire database cannot fit into LLM context.	Aggregation, metadata RAG, hierarchical summarization.
Query cost	Poor SQL can scan huge datasets.	Query validation, partitioning, cost estimation, materialized views.
Schema complexity	Thousands of tables confuse the model.	Metadata catalog and semantic retrieval.
Business semantics	Correct SQL can still answer the wrong business question.	Certified metric definitions and semantic layer.
Latency	Multi-step agent loops can be slow.	Parallel queries, caching, pre-aggregation.
Security	Database may contain sensitive information.	RBAC, masking, row/column security.
Hallucination	LLM may generate invalid SQL or unsupported claims.	SQL validation, evidence binding, deterministic calculations.

Agent loops	Autonomous investigation can become expensive.	Step budgets and query budgets.
Trust	Users need to know why a conclusion was reached.	Query lineage and evidence references.

29 Recommended Proof-of-Concept

A practical student POC should start small.

29.1 Phase 1: Fixed Analytical Database

Use a database containing tables such as:

```
customers
products
orders
order_items
regions
```

Populate it with synthetic data.

29.2 Phase 2: Metadata Retrieval

Create metadata describing:

- Tables.
- Columns.
- Relationships.
- Business definitions.

Add semantic retrieval for schema discovery.

29.3 Phase 3: Text-to-SQL

Implement:

$$\text{Question} \rightarrow \text{Relevant Schema} \rightarrow \text{SQL}$$

Add SQL validation before execution.

29.4 Phase 4: Analysis Agent

Allow the agent to compare:

- Current period vs previous period.
- Regions.
- Products.
- Customer segments.

29.5 Phase 5: Agentic Drill-Down

Introduce the loop:

$$\text{Analyze} \rightarrow \text{Detect Significant Finding} \rightarrow \text{Drill Down} \rightarrow \text{Analyze Again}$$

The loop should stop when sufficient evidence has been collected.

29.6 Phase 6: Agentic Chat

Add conversational follow-up:

User: "Summarize Q2 revenue."
Agent: "Revenue increased 15%."
User: "Why?"
Agent: "APAC and Product X were the largest contributors."
User: "Tell me more about Product X."
Agent: "Product X grew 28%, mainly among enterprise customers."

The conversation state should preserve the analytical context.

30 Example End-to-End Workflow

Consider:

"Why did revenue decline in Q2?"

The workflow may be:

1. Planner identifies the metric, period, and comparison period.
2. Schema Agent retrieves revenue and transaction metadata.
3. Semantic Layer resolves "revenue" to the certified metric.
4. Query Agent creates the initial aggregate query.
5. Guardrail validates the query.
6. Database executes the query.
7. Analysis Agent detects a 12% decline.
8. Insight Agent chooses region as the next dimension.
9. Database calculates regional changes.
10. North America shows a 30% decline.
11. Insight Agent drills into product.
12. Product X explains most of the decline.
13. Insight Agent drills into customer segment.
14. Enterprise customers are identified as the major contributor.

15. Summary Agent produces the final evidence-based response.

The resulting explanation could be:

> Revenue declined 12% in Q2 compared with Q1. The primary driver was a 30% decline in North America, which accounted for most of the overall reduction. Product X contributed approximately 65% of the North American decline, with enterprise customers representing the largest affected segment.

The important distinction is that the answer is generated from a chain of database-backed evidence rather than from the LLM's general knowledge.

31 Design Principles

The following principles should guide implementation.

1. **LLM as planner, not database engine.**
2. **Push computation to the database.**
3. **Retrieve relevant schema rather than exposing everything.**
4. **Use a semantic layer for business meaning.**
5. **Validate generated SQL before execution.**
6. **Use deterministic tools for numerical calculations.**
7. **Bound agentic loops.**
8. **Keep authorization outside the LLM.**
9. **Record evidence and lineage.**
10. **Return conclusions with supporting evidence.**
11. **Start with a controlled POC before introducing full autonomy.**

32 Discussion Questions

Students should consider the following questions when designing their own implementation:

1. What happens if the database contains 10 billion rows?
2. How can the agent discover the correct table among 10,000 tables?
3. How should the system resolve an ambiguous term such as "revenue"?
4. How can we prevent the agent from generating an expensive full-table scan?
5. Should the agent be allowed to execute arbitrary SQL?
6. How should PII be protected?
7. How should different users receive different database views?
8. What happens when two agents produce contradictory findings?

9. How do we evaluate whether the generated SQL is correct?
10. How do we evaluate whether the final explanation is faithful to the data?
11. When should the agent stop investigating?
12. Which operations should be deterministic rather than LLM-generated?
13. How can query lineage be exposed to the user?
14. What should be cached?
15. When does a multi-agent design become unnecessarily complex?

33 Key Takeaway

A scalable database-analysis agent is not:

$$\boxed{\text{Database} \rightarrow \text{LLM} \rightarrow \text{Summary}}$$

A more robust architecture is:

$$\boxed{\text{User} \rightarrow \text{Planner} \rightarrow \text{Metadata/Semantics} \rightarrow \text{Query} \rightarrow \text{Database} \rightarrow \text{Analysis} \rightarrow \text{Evidence} \rightarrow \text{LLM Summary}}$$

For very large databases, the most important engineering principle is to keep large-scale computation inside the data platform and provide the LLM with small, relevant, validated, and explainable results.

The "agentic" capability is then used for deciding *what to investigate next*, rather than for indiscriminately reading data.

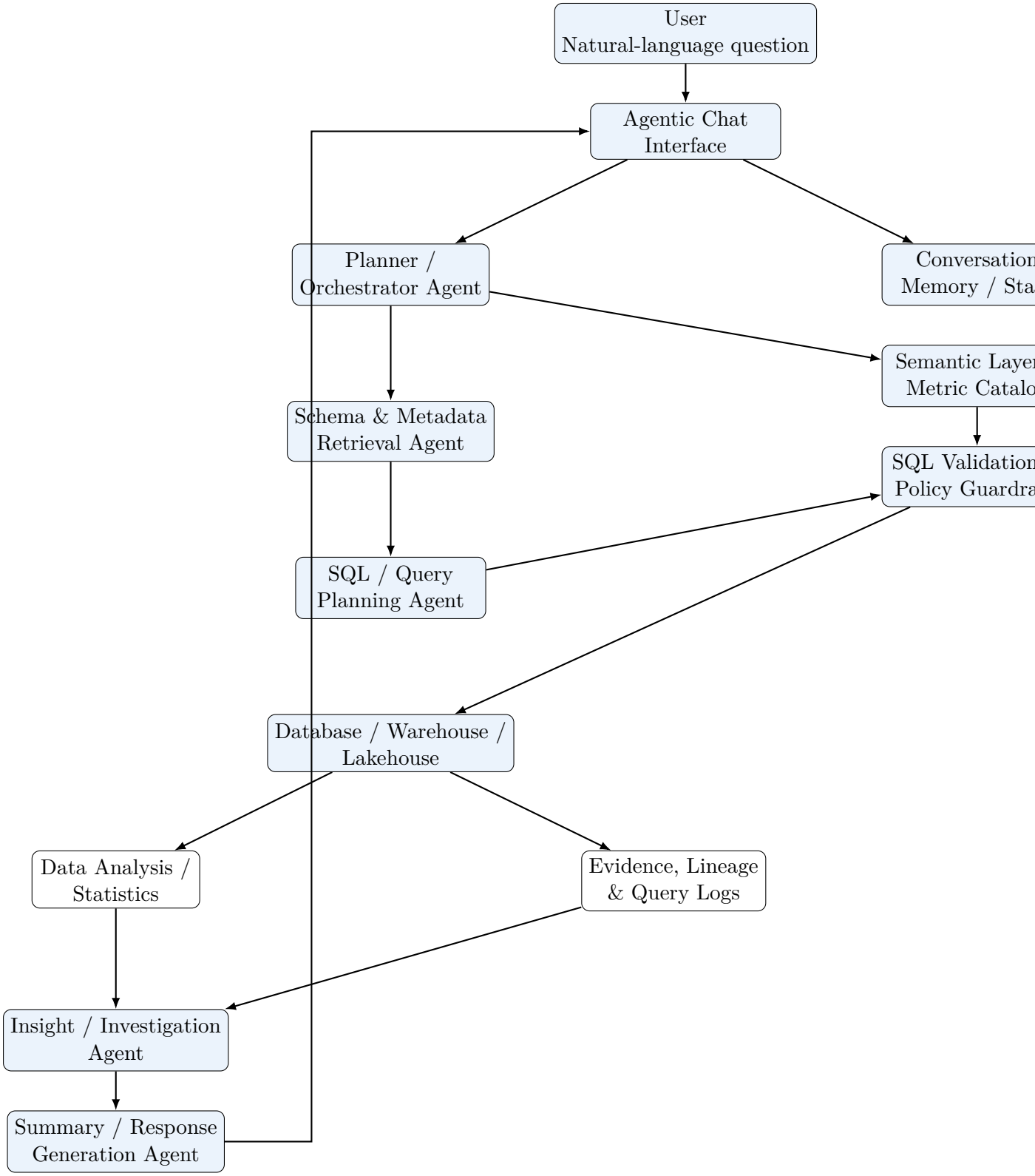

Figure 13: Conceptual architecture of the multi-agent Database Analyzer.

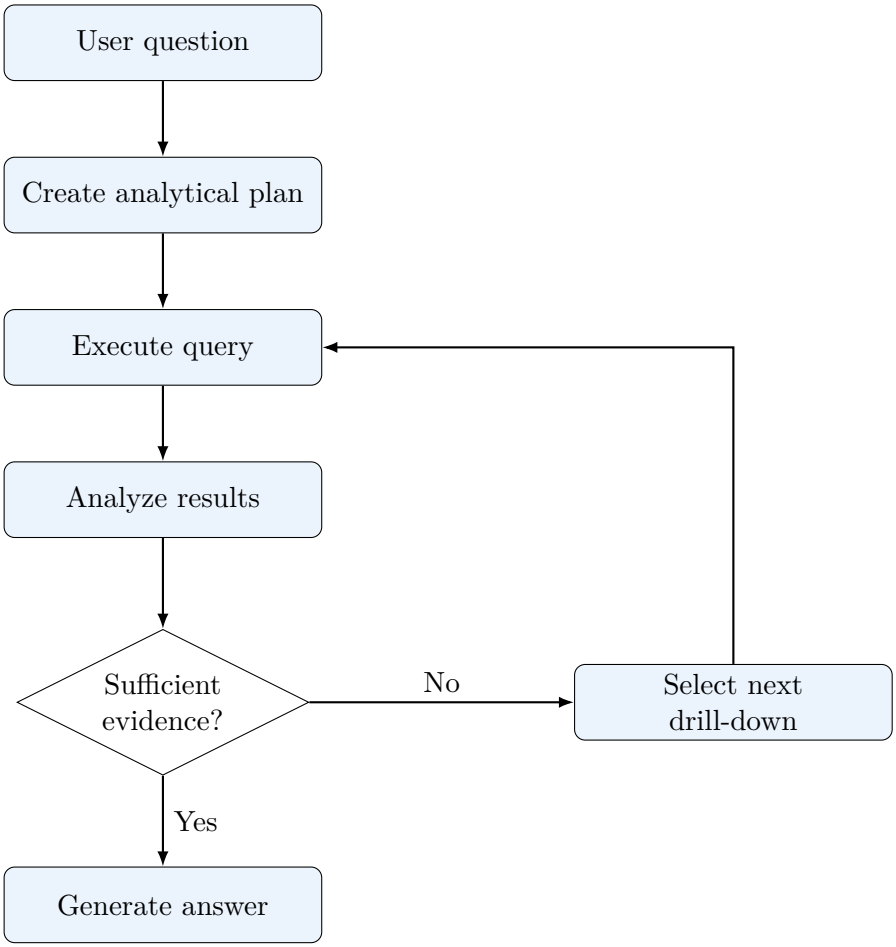

Figure 14: Agentic investigation loop.